

Prueba Técnica Full-Stack: Aplicación Web Agrícola

Importante antes de comenzar

- Tecnología a utilizar: Laravel, Reactjs.
- Para la base de datos, se adjunta un archivo **prueba_tecnica_base_de_datos.html** donde se encuentra el detalle y las indicaciones de las tablas a crear y el entregable. El motor de base de datos se recomienda que sea Microsoft SQL Server, pero se puede utilizar cualquier otro motor de base de datos relacional que se considere oportuno.
- Para el frontend y backend clonar los repositorios proporcionados en su cuenta personal para posterior entrega, se tiene la libertad de realizar ajustes a la plantilla o instalar las librerías necesarias para completar la prueba.
- Se contará con 1 hora para realizar la prueba a partir de la hora definida, el último commit antes de que el tiempo se complete es el que se tomará en cuenta, se recomienda estar atento al tiempo restante una vez iniciada la prueba para evitar inconvenientes.

Requerimientos

HU 1: Autenticación y Protección de Accesos (Login) - Frontend

Como usuario del sistema, **Quiero** poder iniciar sesión con credenciales predefinidas, **Para** acceder de forma segura a las rutas protegidas de la aplicación.

- **Instrucciones (Criterios de Aceptación):**
 - Crear una pantalla de Login visualmente atractiva.
 - Validar estáticamente las credenciales: Usuario *devcassa* / Contraseña *cassa123*.
 - No es necesario conectar el login a la base de datos ni implementar JWT real.
 - Proteger todas las rutas internas de la aplicación; si no hay sesión activa, redirigir al Login.
- **Puntos a evaluar (15 pts):**
 - Protección de rutas (Route Guards/Middlewares en el frontend).
 - Manejo correcto de estados globales de la aplicación (Context, Zustand, Redux, etc.).
 - Limpieza del estado global al ejecutar la acción de "Cerrar sesión".

HU 2: Estructura Base y Navegación - Frontend

Como usuario autenticado, **Quiero** visualizar un menú de navegación lateral, **Para** moverme fácilmente entre los distintos módulos del sistema.

- **Instrucciones (Criterios de Aceptación):**
 - Crear un Layout principal que envuelva las vistas protegidas.
 - Implementar un menú lateral (Sidebar) con las opciones: Dashboard, Haciendas, Responsables.

- Incluir un botón visible para el "Cierre de sesión".
- **Puntos a evaluar (10 pts):**
 - Estructuración de componentes compartidos (Layout).
 - Uso adecuado de la librería de enrutamiento (ej. React Router).

HU 3: Dashboard Principal (Resumen) – Frontend y Backend

Como administrador agrícola, **Quiero** ver un resumen de las entidades activas en el sistema, **Para** tener un panorama rápido de la operación al iniciar sesión.

- **Instrucciones (Criterios de Aceptación):**
 - Ser la pantalla por defecto tras iniciar sesión.
 - Mostrar tres *Cards* (tarjetas) con los siguientes indicadores: Cantidad de Haciendas activas, Cantidad de Lotes activos y Cantidad de responsables activos.
 - Los datos mostrados deben reflejar la información real guardada en la base de datos.
- **Puntos a evaluar (15 pts):**
 - Diseño y maquetación de las tarjetas.
 - Integración con el backend (End-points tipo GET para contadores o agregaciones).

HU 4: Gestión de responsables (CRUD) – Frontend y Backend

Como administrador agrícola, **Quiero** visualizar y administrar el catálogo de responsables, **Para** mantener actualizada la información del personal a cargo.

- **Instrucciones (Criterios de Aceptación):**
 - Crear una vista con el listado de responsables.
 - Permitir la creación, edición y eliminación de responsables.
 - Manejar un campo de "Estatus" (Activo/Inactivo).
- **Puntos a evaluar (15 pts):**
 - Diseño de base de datos y API REST para responsables.
 - Manejo de formularios y validaciones en el frontend.
 - Actualización de la UI tras cada operación exitosa.

HU 5: Gestión de Haciendas y Navegación a Lotes – Frontend y Backend

Como administrador agrícola, **Quiero** administrar el listado de haciendas y acceder a los detalles de cada una, **Para** gestionar los terrenos de forma organizada.

- **Instrucciones (Criterios de Aceptación):**
 - Crear una vista con el listado de Haciendas (con su estatus Activo/Inactivo).
 - Permitir la creación, edición y eliminación de haciendas.
 - Al hacer clic sobre el registro de una hacienda, la aplicación debe navegar a la vista de "Lotes", pasando el contexto de la hacienda seleccionada.
- **Puntos a evaluar (15 pts):**
 - Diseño de API y modelo relacional (Hacienda).
 - Navegación dinámica con parámetros en la URL (ej. /haciendas/:id/lotes).

HU 6: Gestión de Lotes por Hacienda (CRUD) – Frontend y Backend

Como administrador agrícola, **Quiero** administrar los lotes que pertenecen a una hacienda específica, **Para** mantener el control de las divisiones de tierra.

- **Instrucciones (Criterios de Aceptación):**
 - Mostrar el listado de lotes correspondientes *únicamente* a la hacienda seleccionada previamente.
 - Permitir la creación, edición y eliminación de lotes vinculados a esa hacienda.
- **Puntos a evaluar (10 pts):**
 - Manejo de llaves foráneas y consultas filtradas en el backend.
 - Manejo del contexto/estado en el frontend (saber a qué hacienda se le está agregando el lote).

Consideraciones Generales y Rúbrica de Puntuación (20 pts)

Estos puntos son transversales a toda la aplicación y se evaluarán basándose en el resultado final del proyecto:

Criterio a Evaluar	Detalle de la Expectativa	Puntuación
Prevención de Errores (Modales de Confirmación)	Toda acción de eliminar (Haciendas, Lotes o Responsables) debe interceptarse con un modal o alerta emergente para confirmar la acción antes de ejecutarla.	5 pts
Diseño Moderno (UI)	Uso de sistemas de diseño coherentes (espaciados, tipografías, paleta de colores). Se valora el uso de TailwindCSS o librerías de componentes (HeroUI, Shadcn, etc).	5 pts
Buena Experiencia de Usuario (UX)	Feedback visual al usuario: estados de carga (<i>loading/spinners</i>), notificaciones de éxito o error (<i>toast/snackbars</i>), y manejo adecuado de estados vacíos (cuando no hay registros).	5 pts
Buenas Prácticas de Código	Código limpio, modularización de componentes, nomenclatura descriptiva en inglés o español (pero consistente), separación de responsabilidades y uso adecuado de variables de entorno.	5 pts